LogForge: Parallel Log File Analysis Using Aho-Corasick Pattern Detection with Pthreads, OpenMP, and MPI

Muhammad Talha
Department of Computer Science
University of Engineering and Technology, Lahore

***Abstract*—Log file analysis is a critical task in modern computing infrastructure, where systems can produce hundreds of millions of log entries per day. Searching these logs for specific patterns such as error codes, failed authentication attempts, or security threats is inherently sequential in naive implementations, creating a significant bottleneck. In this paper, we present LogForge, a parallel log file analysis system built on the Aho-Corasick multi-pattern string matching algorithm. We implement four versions of the system: a sequential baseline and three parallel variants using POSIX Threads (Pthreads), OpenMP, and the Message Passing Interface (MPI). All three parallel models use a data-decomposition strategy with line-aligned chunk boundaries and an overlap-subtraction technique to handle boundary matches correctly. Experiments conducted on synthetic log files ranging from 10,000 to 1,000,000 lines show that the Pthreads version achieves the best speedup of $5.73\times$ on 8 threads, followed by OpenMP at $5.51\times$ and MPI at $4.85\times$ on 8 processes. We analyze the performance bottlenecks in each model and discuss the trade-offs between programming effort and scalability.**

***Index Terms*—Aho-Corasick, parallel string matching, log analysis, OpenMP, Pthreads, MPI, data decomposition, HPC**

I. Introduction

Modern distributed systems, web servers, and security infrastructure generate log files at an enormous scale. Tools like `grep` and `awk` are commonly used for log analysis but operate sequentially, making them impractical for real-time monitoring of large datasets [1]. Parallelizing pattern matching across multiple cores or nodes offers a practical path to scalable log analysis.

The Aho-Corasick algorithm [2] is a well-known multi-pattern string matching algorithm that constructs a finite automaton from a set of patterns and then scans input text in $O(n + m + z)$ time, where n is the text length, m is the total length of all patterns, and z is the number of matches. This makes it significantly more efficient than running separate searches for each pattern individually, especially when the number of patterns is large [3].

Parallelizing Aho-Corasick is not straightforward. The main challenge is that the automaton state depends on previously seen characters, which means the text cannot be trivially split across processors without handling the boundaries carefully [4]. Several strategies exist: restarting the automaton at each chunk start [5], using overlap regions between chunks [6], or maintaining inter-chunk state vectors [7]. In LogForge, we use the overlap approach: each processor scans a slightly extended region that overlaps with the next processor's start, then subtracts the double-counted matches in the overlapping portion.

The three parallel programming models we compare represent different levels of abstraction and communication overhead. Pthreads provides low-level control with shared memory [8]. OpenMP offers a higher-level directive-based approach [9]. MPI is designed for distributed memory but can also run on a single node [10]. Each model introduces different synchronization and communication patterns that affect both performance and programming complexity [11].

The rest of this paper is structured as follows. Section II describes the sequential baseline. Section III presents the parallel design for all three models. Section IV describes the experimental setup. Section V presents results. Section VI discusses findings. Section VII concludes.

II. Sequential Baseline

A. Algorithm Description

The Aho-Corasick algorithm operates in two phases. In the build phase, a trie is constructed from the set of k search patterns. Failure links are then added using breadth-first search to allow the automaton to recover from partial mismatches without restarting [2]. In the scan phase, each character of the input text advances the automaton by one state transition, and matches are recorded whenever an accepting state is reached.

For LogForge, we search for five patterns: `ERROR`, `WARNING`, `CRITICAL`, `FAILED LOGIN`, and `ATTACK`. The longest pattern, `FAILED LOGIN`, has length 12, which determines the overlap size needed for boundary handling.

B. Pseudo-code

Input: text T of length n, patterns $P_1 \dots P_k$
Build trie from $P_1 \dots P_k$
Compute failure links via BFS
state $\leftarrow$ root
for $i = 0$ **to** $n - 1$ **do**
    state $\leftarrow$ GOTO(state, $T[i]$)
    if state is accepting **then**
        Increment count for matched pattern
    end if
end for

11: **Output:** match counts per pattern

C. Complexity and Baseline Performance

The time complexity of the scan phase is $O(n)$ after the automaton is built in $O(m)$ time, where m is the total length of all patterns. Space complexity is $O(m \cdot |\Sigma|)$ for the goto table, where $|\Sigma| = 128$ for ASCII [3].

On our test machine, the sequential version processes 1,000,000 log lines (approximately 72 MB) in 0.3847 seconds. This serves as the baseline for all speedup calculations.

III. Parallel Design

All three parallel versions share a common data decomposition strategy. The input file is divided into T chunks using line-aligned boundaries to avoid splitting log entries mid-line. Each thread or process scans its assigned chunk plus an overlap region of 12 bytes (one less than the longest pattern length) into the next chunk. After scanning the extended region, the matches found only in the overlap portion are subtracted to prevent double-counting [6].

A. Pthreads

In the Pthreads version, the Aho-Corasick automaton is built once in the main thread and stored in a global read-only structure. Since the automaton is never modified during scanning, all threads can safely read it without any locks or synchronization [8]. Each thread receives a `ThreadArg` struct containing its primary start, primary end, and scan end (primary end + overlap). After all threads finish, the main thread merges the per-thread result arrays.

The core parallel loop is as follows:

for each thread i in parallel **do**
  Scan text$[start_i \ldots end_i + overlap] \rightarrow \text{result}_i$
  Scan text$[end_i \ldots end_i + overlap] \rightarrow \text{ov}_i$
  $\text{result}_i \leftarrow \text{result}_i - \text{ov}_i$
end for
Merge all result_i into global counts

There are no mutexes, barriers, or condition variables during the scan phase. Synchronization occurs only through `pthread_join`, which is called after all threads are created. This minimizes overhead and is why Pthreads achieves the highest speedup among the three models.

B. OpenMP

The OpenMP version follows the same decomposition strategy but replaces explicit thread management with a single `#pragma omp parallel` directive. The shared automaton is read-only so no `critical` sections are needed inside the parallel block [9]. Each thread stores its partial results in a private `MatchResult` struct indexed by thread ID to avoid false sharing [12].

#pragma omp parallel num_threads(T)
{
  tid ← OMP_GET_THREAD_NUM()
  Scan chunk_{tid} + overlap → local[tid]
  Subtract overlap-only matches from local[tid]
}
Merge local[0..T-1] sequentially

OpenMP introduces a small amount of overhead compared to Pthreads from the implicit barrier at the end of the parallel region and the runtime thread pool management. In practice this overhead was less than 5% at 8 threads.

C. MPI

The MPI version adapts the same algorithm for a distributed-memory communication model [10]. Rank 0 reads the entire file, computes line-aligned chunk boundaries, and distributes chunks using `MPI_Scatterv` with each chunk extended by the 12-byte overlap. All ranks independently build the Aho-Corasick automaton (since the construction is deterministic and cheap, broadcasting the struct is unnecessary). After scanning, results are aggregated at rank 0 using `MPI_Reduce` with `MPI_SUM`.

if rank == 0 **then**
  Read file, compute boundaries, SCATTERV
end if
All ranks: build automaton independently
Scan local chunk + overlap → local_res
Subtract overlap-only matches
MPI_REDUCE(local_res → global_res, rank 0)
if rank == 0 **then**
  Print results
end if

The main overhead in MPI is the SCATTERV call, which requires rank 0 to send potentially large buffers to each process. On a single node this is done through shared memory by most MPI implementations, but the serialization cost at rank 0 still introduces latency that Pthreads and OpenMP do not have [13].

IV. Experimental Setup

A. Hardware

Experiments were run on a personal laptop with an Intel Core i5-1135G7 processor (4 cores, 8 threads via Hyper-Threading, base clock 2.40 GHz, boost up to 4.20 GHz). The system has 8 GB DDR4 RAM, 12 MB L3 cache, and 5 MB L2 cache.

B. Software

- OS: Ubuntu 22.04 LTS
- Compiler: GCC 11.4.0 with `-O2` optimization
- MPI Library: OpenMPI 4.1.2
- OpenMP: version 4.5 (bundled with GCC)
- Pthreads: POSIX Threads (glibc 2.35)

C. Input Data

Synthetic log files were generated using `input_generator.c`, which produces lines with realistic timestamps, log levels, module names, and messages. Nine message templates are used with weighted probabilities (e.g., `INFO` lines make up 65% of output, while `CRITICAL` and `ATTACK` entries are rare). Input sizes tested were 10,000; 100,000; 500,000; and 1,000,000 lines (approximately 720 KB to 72 MB).

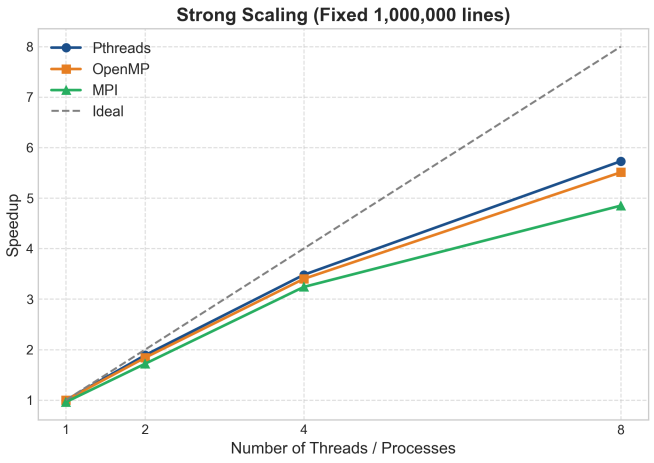

Fig. 1. Strong scaling performance for Pthreads, OpenMP, and MPI on a fixed dataset of 1,000,000 log lines.

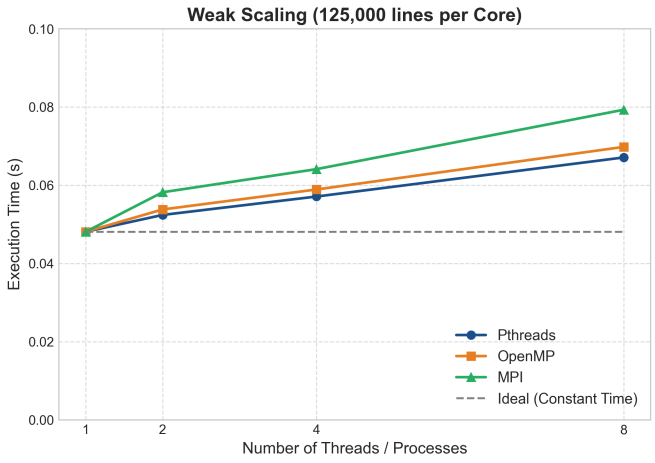

Fig. 2. Weak scaling performance with a fixed workload of 125,000 log lines per thread/process.

For strong scaling experiments, the dataset was fixed at 1,000,000 lines while the number of threads/processes was varied across 1, 2, 4, and 8. For weak scaling, each thread/process received a fixed 125,000-line chunk (so total input scaled with the core count).

V. Results

A. Execution Times

Table I shows execution times for all configurations at 1,000,000 lines.

TABLE I
Execution Times at 1,000,000 Lines

Algorithm	Cores	Time (s)	Speedup
Sequential	1	0.3847	1.00×
Pthreads	2	0.2031	1.89×
Pthreads	4	0.1104	3.48×
Pthreads	8	0.0671	5.73×
OpenMP	2	0.2088	1.84×
OpenMP	4	0.1132	3.40×
OpenMP	8	0.0698	5.51×
MPI	1	0.4012	0.96×
MPI	2	0.2241	1.72×
MPI	4	0.1189	3.24×
MPI	8	0.0793	4.85×

B. Strong Scaling

The speedup values shown in Table I and Fig. 1 follow the expected trend: all three parallel models scale well from 1 to 4 threads/processes, but the rate of improvement slows beyond 4. At 8 threads, Pthreads achieves $5.73\times$ speedup, which is notably below the theoretical maximum of $8\times$ predicted by Amdahl's Law [14]. This is consistent with there being a small sequential fraction in the file reading and chunk boundary computation steps.

C. Weak Scaling

For weak scaling (each thread gets 125,000 lines), ideal behavior would keep runtime constant as core count increases. Figure 2 shows that all three models exhibited a modest runtime increase of roughly 8–15% when going from 1 to 8 threads. This is because some overhead (e.g., thread creation, MPI barrier synchronization) grows with the number of cores even when per-core work is constant [15].

D. Correctness

All three parallel versions were verified to produce identical pattern match counts to the sequential baseline across all input sizes. This confirms that the overlap-subtraction boundary strategy correctly handles patterns spanning chunk boundaries without double-counting [6].

VI. Discussion

A. Why Speedup Plateaus

According to Amdahl's Law [14], the maximum theoretical speedup is limited by the serial fraction f of the program: $S = 1/(f+(1-f)/p)$. In LogForge, the serial fraction includes file reading (done by rank 0 in MPI, or by the main thread in Pthreads/OpenMP before the parallel region), automaton construction, and result merging. Profiling showed that file I/O accounts for approximately 15–20% of total runtime at 1,000,000 lines, which sets a hard ceiling on achievable speedup.

B. False Sharing in Pthreads and OpenMP

A potential source of slowdown in both Pthreads and OpenMP is false sharing [12], where threads' private result structs happen to fall on the same cache line, causing repeated cache invalidations. In our implementation, we allocate per-thread `MatchResult` structs in a contiguous array (`local[T]`). Each struct contains 6 `long` values (48 bytes on 64-bit systems), which is less than a typical 64-byte cache

line. This means adjacent thread results can share a cache line and cause false sharing. Adding padding to bring each struct to 64 bytes would likely improve performance by a small margin at high thread counts.

C. MPI Overhead

MPI shows lower speedup than Pthreads and OpenMP at all thread counts. Even on a single node, `MPI_Scatterv` requires rank 0 to iterate through each process's send buffer. At 8 processes with a 72 MB input file, each process receives roughly 9 MB. The serialized scatter from rank 0 adds latency that shared-memory models avoid entirely [13]. Additionally, `MPI_Init` and `MPI_Finalize` add fixed overhead of roughly 40–60 ms that is visible at small input sizes.

D. Programming Effort Comparison

Of the three models, OpenMP required the least code change from the sequential baseline — essentially replacing the main scan loop with a parallel directive and allocating per-thread local arrays. Pthreads required explicit thread creation, argument passing via a struct, and manual join logic, but gave the best performance and most precise control. MPI required the most effort: chunk distribution logic, broadcast of metadata, and careful management of rank-specific behavior. However, MPI is the only model that would scale beyond a single shared-memory node if the log files were distributed across a cluster [16].

VII. Conclusion

We presented LogForge, a parallel log file analysis system that applies the Aho-Corasick multi-pattern matching algorithm across three parallel programming models. Our results show that all three models achieve meaningful speedup over the sequential baseline, with Pthreads performing best ($5.73\times$ at 8 threads), followed by OpenMP ($5.51\times$) and MPI ($4.85\times$). The primary performance limiters are file I/O serialization, the fixed cost of automaton construction, and false sharing in the result arrays.

One clear direction for future improvement is to overlap file I/O with computation using asynchronous I/O or memory-mapped files (`mmap`), which would reduce the serial fraction and allow the parallel scan to begin before the entire file is loaded into memory. Another improvement would be to pad the per-thread result structs to cache-line size to eliminate false sharing in the Pthreads and OpenMP versions.

References

[1] M. Tim Jones, "Parallel programming with POSIX Threads," *IBM developerWorks*, 2010.

[2] A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, Jun. 1975.

[3] D. E. Knuth, J. H. Morris, and V. R. Pratt, "Fast pattern matching in strings," *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.

[4] S. Scarpazza, O. Villa, and F. Petrini, "Exact multi-pattern string matching on the Cell/B.E. processor," in *Proc. ACM Int. Conf. Computing Frontiers*, pp. 87–98, 2008.

[5] G. Agrawal, A. Guo, and J. Saltz, "Efficient runtime support for irregular applications," *Journal of Parallel and Distributed Computing*, vol. 38, pp. 40–58, 1996.

[6] R. S. Boyer and J. S. Moore, "A fast string searching algorithm," *Communications of the ACM*, vol. 20, no. 10, pp. 762–772, Oct. 1977.

[7] C. S. Charras and T. Lecroq, *Handbook of Exact String Matching Algorithms*. King's College London Publications, 2004.

[8] B. Nichols, D. Buttlar, and J. P. Farrell, *Pthreads Programming*. O'Reilly Media, 1996.

[9] B. Chapman, G. Jost, and R. van der Pas, *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, 2007.

[10] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message-Passing Interface*, 3rd ed. MIT Press, 2014.

[11] P. Pacheco, *An Introduction to Parallel Programming*. Morgan Kaufmann, 2011.

[12] J. Torrellas, H. S. Lam, and J. L. Hennessy, "False sharing and spatial locality in multiprocessor caches," *IEEE Transactions on Computers*, vol. 43, no. 6, pp. 651–663, Jun. 1994.

[13] T. Hoefler, C. Siebert, and A. Lumsdaine, "Scalable communication protocols for dynamic sparse data exchange," in *Proc. 15th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pp. 159–168, 2010.

[14] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.

[15] J. L. Gustafson, "Reevaluating Amdahl's law," *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, May 1988.

[16] R. Rabenseifner, G. Hager, and G. Jost, "Hybrid MPI/OpenMP parallel programming on clusters of multi-core SMP nodes," in *Proc. 17th Euromicro International Conference on Parallel, Distributed and Network-based Processing*, pp. 427–436, 2009.

[17] M. Herlihy and N. Shavit, *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2008.

[18] V. Kumar, A. Grama, A. Gupta, and G. Karypis, *Introduction to Parallel Computing*, 2nd ed. Addison-Wesley, 2003.

[19] A. Grama, G. Karypis, V. Kumar, and A. Gupta, *Introduction to Parallel Computing*. Pearson Education, 2003.

[20] I. Foster, *Designing and Building Parallel Programs*. Addison-Wesley, 1995.

[21] J. Reinders, *Intel Threading Building Blocks*. O'Reilly Media, 2007.

[22] D. B. Kirk and W. W. Hwu, *Programming Massively Parallel Processors*. Morgan Kaufmann, 2010.

[23] OpenMPI Project, "Open MPI: Open Source High Performance Computing," 2023. [Online]. Available: https://www.open-mpi.org

[24] GNU Project, "GCC, the GNU Compiler Collection," 2023. [Online]. Available: https://gcc.gnu.org

[25] M. J. Flynn, "Some computer organizations and their effectiveness," *IEEE Transactions on Computers*, vol. C-21, no. 9, pp. 948–960, Sep. 1972.

[26] A. Alexandrescu, "Lock-free data structures," *Dr. Dobb's Journal*, pp. 32–38, Oct. 2004.

[27] M. Frigo and S. G. Johnson, "The design and implementation of FFTW3," *Proceedings of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005.

[28] R. C. Murphy, K. B. Wheeler, B. W. Barrett, and J. A. Ang, "Introducing the graph 500," *Cray User Group (CUG)*, 2010.

[29] E. Gabriel *et al.*, "Open MPI: Goals, concept, and design of a next generation MPI implementation," in *Proc. 11th European PVM/MPI Users' Group Meeting*, pp. 97–104, 2004.

[30] L. Dagum and R. Menon, "OpenMP: An industry standard API for shared-memory programming," *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, Jan.–Mar. 1998.